



## **Tecnologías de Desarrollo de Aplicaciones Empresariales sobre Internet**

Máster en Ingeniería Informática

# **Fase 1.2 Domain-Driven Design**

**Autor:** Miguel Martín  
**Fecha:** 9 de junio de 2026

## 1. Agregates y Aggregate Roots

A continuación, se identifican los Aggregates del sistema, recordando que un aggregate está formado por un **conjunto cohesionado de elementos con un ciclo de vida común**, dominado por su Aggregate Root. La tipología de cada elemento ha sido justificada basándose en su mutabilidad y su hilo de continuidad.

### 1.1. Aggregate: Catálogo

Este aggregate constituye un conjunto coherente de información sobre un mismo concepto: la oferta de contenido de la plataforma. Las clases `Serie`, `Temporada`, `Capitulo` y `CategoriaSerie` poseen un ciclo de vida común dominado por el aggregate root. Por otra parte se incluye la clase `Persona`, la cual no tendría sentido que existiera en el sistema sin la clase `Serie`, ya que solo se utiliza para describir a los Actores y Creadores de dicha serie.

- **Aggregate Root: Serie**
- **Elementos que lo conforman:**
  - **Serie** (*Entity*): Es claramente una entidad, ya que se trata de objetos con una identidad definida clara, y que es necesario preservar con claridad en el sistema. Debemos diferenciar una serie concreta de cualquier otra. Además, necesita actuar como el punto de acceso a partir del cual se recuperan los otros elementos, monitorizando su ciclo de vida.
  - **Temporada y Capitulo** (*Entity*): Son entidades locales al agregado. Poseen un hilo de continuidad; por ejemplo, si un creador actualiza el enlace de vídeo meses después de su publicación o añade un capitulo, actualizamos la misma entidad en lugar de crear una nueva. Su existencia evoluciona con el tiempo.
  - **Persona** (*Entity*): Lo considero Entity local porque nos interesa el hilo de continuidad del individuo real en el sistema. Tienen una identidad única y bien definida (si cambian legalmente su nombre, la actualización debe reflejarse históricamente en el reparto de las series).
  - **CategoriaSerie** (*Value Object*): Se trata simplemente de información con un formato concreto que se ha extraído de la clase Serie por conveniencia para no contaminarla. Pueden existir varias instancias duplicadas (muchas series siendo "GOLD") sin que ello suponga ningún problema para el sistema. Carece de identidad; si una serie cambia de tarifa, simplemente sustituimos este Value Object por otro, no lo editamos.

### 1.2. Aggregate: Usuario

Este aggregate es el encargado de gestionar la identidad del suscriptor, sus datos de acceso y su seguimiento. Considero que las clases que debe contener son `Usuario`, `TipoSuscripcion` y `RegistroSerieUsuario`, ya que están altamente relacionadas y deben tratarse como una unidad. La clase `TipoSuscripcion` se incluye dentro de este grupo porque define la forma de cobrar del usuario. Además, el `RegistroSerieUsuario` se aloja aquí argumentando que es el propio usuario el máximo responsable de tener el conocimiento y el control sobre cuál es el último capítulo que ha visto de cada serie. No tiene sentido que sobreviva un registro de seguimiento cuando se ha dado de baja al usuario del sistema.

- **Aggregate Root: Usuario**

- **Elementos que lo conforman:**

- **Usuario** (*Entity*): Es claramente una entidad, pues los usuarios necesitan ser identificados de manera unívoca. Su identidad perdura más allá de sus atributos actuales (si el usuario cambia su cuenta bancaria, el historial sigue siendo suyo). Es el *Aggregate Root* al controlar el ciclo de vida de los registros de lectura.
- **TipoSuscripcion** (*Value Object*): Al igual que la categoría de las series, se trata de simples datos que pueden estar perfectamente duplicados en el sistema para multitud de usuarios distintos sin problema. Es información puramente descriptiva e inmutable.
- **RegistroSerieUsuario** (*Entity*): Actúa como un marcador de lectura activo. Es una entidad porque tiene cambios de estado claros que necesitan ser adecuadamente monitorizados y gestionados (el objeto muta actualizando su estado interno cada vez que el usuario avanza de capítulo).

### 1.3. Aggregate: Facturación

Este aggregate contiene las clases *Factura* y *Visualizacion*. Se ha separado del aggregate de *Usuario* para dividir responsabilidades y porque, aunque un usuario se dé de baja, las facturas por motivos fiscales y de auditoría suelen requerir un ciclo de vida independiente.

- **Aggregate Root: Factura**

- **Elementos que lo conforman:**

- **Factura** (*Entity*): Las facturas necesitan ser identificadas de manera unívoca dentro del sistema. Tienen cambios de estado claros (abierta, en facturación, cobrada) que necesitan ser adecuadamente monitorizados y gestionados a lo largo de su ciclo de vida.
- **Visualización** (*Entity*): Es una entidad porque cada visualización concreta tiene una identidad única y bien definida. Necesitamos distinguir dos visualizaciones distintas en todo momento para conocer con claridad el historial de cobros y poder auditar incidencias concretas (por ejemplo, reembolsar una visualización fallida modificando su precio cobrado individual).

## 2. Desviaciones de las Reglas Estrictas de DDD

El diseño propuesto toma decisiones arquitectónicas que rompen ciertas reglas del paradigma DDD en favor de la eficiencia computacional en escenarios de alto volumen de datos. Las desviaciones son:

- **Acceso Directo a Entidades Internas de otro Agregado:** La teoría pura de Domain-Driven Design establece que una parte interna de un aggregate no puede referenciar otra parte interna de un aggregate diferente. Todo el acceso a un capítulo debería pasar obligatoriamente a través de la *Aggregate Root* (*Serie*).

Sin embargo, un ejemplo claro de desviación en nuestro modelo es la clase *RegistroSerieUsuario* (perteneciente al agregado *Usuario*), la cual mantiene una referencia de memoria directa hacia *Capitulo* (perteneciente al agregado *Catálogo*) para establecer cuál es el último visualizado. De igual forma, la colección *capitulosVistos* en *Usuario* apunta directamente a *Capitulo*.

Esta desviación es una decisión de diseño consciente. Si siguiésemos las reglas estrictas, para saber por dónde debe continuar viendo una serie el usuario, tendríamos que obtener

el identificador de la serie, recuperar el root **Serie** completo, y recorrer iterativamente sus colecciones internas de *Temporadas*  $\rightarrow$  *Capítulos* hasta dar con el correcto. Permitir esta referencia cruzada directa optimiza drásticamente el caso más frecuente (cargar la pantalla de inicio del usuario) ahorrando consultas complejas a la base de datos.

### 3. Modelo de Dominio UML

El siguiente diagrama UML ilustra las entities, value objects y sus relaciones.

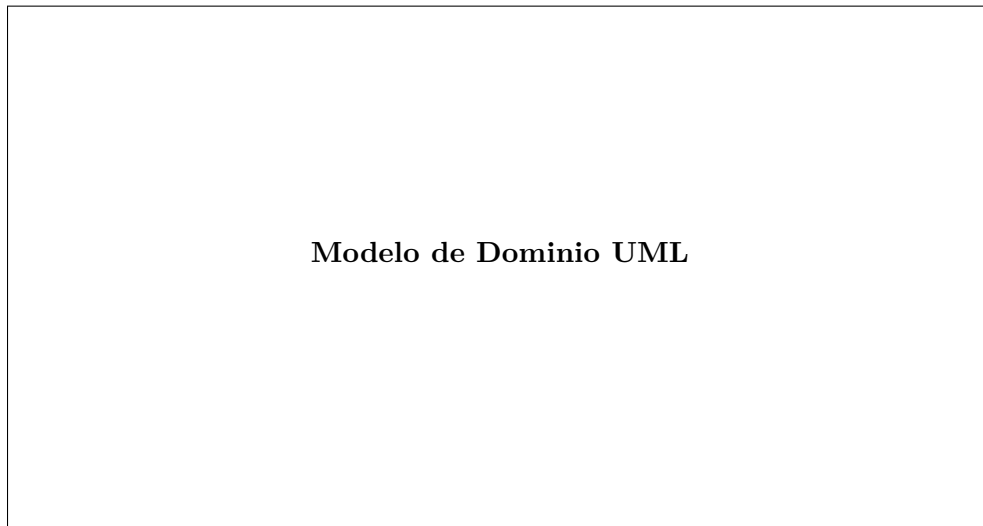


Figura 1: Modelo de Dominio UML.